

Dossier de Conception

Application de Gestion de Bibliotheque Universitaire

Universite Norbert Zongo (UNZ) — Mai 2026

1. Cahier des Charges

1.1 Presentation du projet

L'Universite Norbert Zongo souhaite moderniser sa bibliotheque en developpant une application web complete de gestion de bibliotheque universitaire. Cette application doit permettre de gerer efficacement les ressources documentaires, les emprunts, les reservations et les interactions entre les differents acteurs.

1.2 Acteurs du systeme

Acteur	Role
Etudiant	Consulte le catalogue, recherche des ouvrages, emprunte directement depuis l'interface, reserve, con
Bibliothecaire	Gere le catalogue, les emprunts, les retours, les penalites et les notifications
Administrateur	Gere les utilisateurs, les statistiques, les configurations et les rapports

1.3 Exigences Fonctionnelles

2. Gestion du catalogue : ajout, modification, suppression d'ouvrages avec categories et rayons
3. Recherche avancee : par titre, auteur, ISBN, mots-cles, categorie
4. Emprunt direct par l'etudiant depuis l'interface et retour d'ouvrages par le bibliothecaire
5. Reservation d'ouvrages quand tous les exemplaires sont empruntes
6. Gestion des penalites de retard (100 FCFA/jour standard, 50 FCFA/jour boursier)
7. Notification automatique par email : confirmation emprunt, rappel retour J-2, disponibilite
8. Consultation de l'historique des emprunts par l'etudiant
9. Gestion des utilisateurs : inscription avec INE @unz.bf, authentication JWT, 3 roles
10. Generation de rapports et statistiques : graphiques par mois et par categorie, taux de retard
11. Gestion des categories et rayons de la bibliotheque

1.4 Exigences Non Fonctionnelles

Exigence	Valeur cible
Securite	Authentication JWT, protection donnees personnelles, roles distincts
Performance	Temps de reponse < 2 secondes pour les operations courantes
Disponibilite	>= 99%
Interface	Responsive (ordinateur et mobile), 3 interfaces distinctes par role
Maintenabilite	Principes SOLID, 4 Design Patterns documentes
Versionnement	Git + GitHub avec messages de commit descriptifs

1.5 User Stories

ID	User Story	Acteur	Priorite	Points
US01	En tant qu'etudiant, je veux m'inscrire avec mon INE et mon email @unz.bf afin d'accéder a la bibliotheque	Etudiant	Haute	5
US02	En tant qu'utilisateur, je veux me connecter avec mon INE/mdp afin d'accéder a mon espace	Utilisateur	Moyenne	3

US03	En tant que bibliothecaire, je veux ajouter/modifier un ouvrage avec une date	Bibliothecaire	Haute	4
US04	En tant qu'etudiant, je veux consulter le catalogue en fonction de la disponibilité	Etudiant	Moyenne	3
US05	En tant qu'etudiant, je veux rechercher un ouvrage en fonction de l'auteur, ISBN ou categorie	Etudiant	Basse	4
US06	En tant qu'etudiant, je veux emprunter directement le catalogue	Etudiant	Moyenne	5
US07	En tant que bibliothecaire, je veux enregistrer le retour d'un ouvrage	Bibliothecaire	Moyenne	3
US08	En tant qu'etudiant, je veux reserver un ouvrage indisponible et etre notifié	Etudiant	Moyenne	4
US09	En tant que bibliothecaire, je veux voir les penalites de la bibliothèque	Bibliothecaire	Moyenne	3
US10	En tant qu'etudiant, je veux consulter mon historique de prêts et réservations	Etudiant	Moyenne	2
US11	En tant qu'etudiant, je veux recevoir des emails de confirmation et rappels	Etudiant	Basse	4
US12	En tant qu'administrateur, je veux gerer les utilisateurs	Admin	Basse	4
US13	En tant qu'administrateur, je veux voir les statistiques	Admin	Basse	4
US14	En tant que bibliothecaire, je veux gerer les categories	Bibliothecaire	Basse	2
US15	En tant qu'administrateur, je veux generer des rapports de retards	Admin	Basse	3

2. Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation presente les interactions entre les 3 acteurs et le systeme.
Chaque acteur dispose d'un espace dedie avec des fonctionnalites specifiques.

Acteur	Cas d'utilisation
Etudiant	S'inscrire, Se connecter, Consulter catalogue, Rechercher ouvrage, Emprunter directement, Reserver
Bibliothecaire	Se connecter, Gerer catalogue (ajout/modif/suppression), Enregistrer emprunt, Enregistrer retour, Voir
Administrateur	Se connecter, Gerer utilisateurs (3 roles), Voir statistiques et graphiques, Generer rapports, Configurer

Relations entre cas d'utilisation :

- <> : Emprunter inclut Verifier disponibilite
- <> : Reserver inclut Verifier indisponibilite
- <> : Se connecter inclut Generer token JWT
- <> : Emprunter peut etendre vers Notification email confirmation
- <> : Retourner peut etendre vers Calcul penalite si retard

3. Diagramme de Classes

Le diagramme de classes definit la structure de l'application avec les entites JPA et leurs relations.

Classe	Attributs principaux	Relations
Utilisateur	id, nom, prenom, email, motDePasse, role	Classe abstraite de Etudiant et Bibliothecaire
Etudiant	numeroEtudiant (INE), filiere, niveau	Herite de Utilisateur, 1-N Emprunts, 1-N Reservations
Bibliothecaire	matricule, service	Herite de Utilisateur
Ouvrage	id, titre, auteur, isbn, description, anneePublication	1-N Categories, 1-N Emprunts, 1-N Reservations
Categorie	id, nom, description, rayon	1-N Ouvrages
Emprunt	id, dateEmprunt, dateRetourPrevue, dateRetourReelle, statut	1-N Ouvrages
Reservation	id, dateReservation, statut (EN_ATTENTE/CONFIRME/ANNULE)	1-N Ouvrages

Enumeration StatutEmprunt : EN_COURS, EN_RETARD, RETOURNE

Enumeration Role : ETUDIANT, BIBLIOTHECAIRE, ADMINISTRATEUR

4. Diagrammes de Sequence

4.1 Sequence — Connexion

Etape	Acteur	Action
1	Utilisateur	Saisit email et mot de passe sur login.html
2	Frontend	Envoie POST /api/auth/login avec {email, motDePasse}
3	AuthController	Recoit la requete et appelle la verification
4	UtilisateurRepository	Recherche l'utilisateur par email en base
5	AuthController	Compare le mot de passe (BCrypt ou plain text)
6	JwtUtil	Genere un token JWT avec email et role, expire en 24h
7	Frontend	Recoit {token, role, nom, prenom, id} et redirige vers l'espace correspondant

4.2 Sequence — Emprunt par l'etudiant

Etape	Acteur	Action
1	Etudiant	Clique sur le bouton Emprunter dans le catalogue
2	Frontend	Envoie POST /api/emprunts avec {etudiantId, ouvrageId}
3	EmpruntController	Verifie le token JWT et le role ETUDIANT
4	EmpruntService	Verifie que l'ouvrage a des exemplaires disponibles
5	EmpruntService	Cree l'emprunt avec dateRetour = aujourd'hui + 14 jours
6	OuvrageRepository	Decremente exemplairesDisponibles de 1
7	NotificationService	Envoie email de confirmation a l'etudiant
8	Frontend	Affiche confirmation et rafraichit le catalogue

4.3 Sequence — Reservation

Etape	Acteur	Action
1	Etudiant	Clique sur Reserver (ouvrage avec exemplairesDisponibles = 0)
2	Frontend	Affiche modal de confirmation et envoie POST /api/reservations
3	ReservationService	Verifie qu'il n'existe pas deja une reservation active
4	ReservationService	Cree la reservation avec statut EN_ATTENTE
5	NotificationService	Envoie email de confirmation de reservation
6	Frontend	Affiche confirmation et met a jour Mes Reservations

5. Conception Architecturale (4+1 Vues)

Vue	Description detaillee
Vue Logique	Architecture en 4 couches : (1) Presentation - HTML5/CSS3/JS responsive, 3 interfaces (login, admin, client), (2) Service - API REST, (3) Persistence - JPA, (4) Model - entites, c
Vue Developpement	Packages Java : controller/ (API REST), service/ (logique metier), repository/ (JPA), model/ (entites), c
Vue Processus	Flux d'execution : Requete HTTP → Spring Security (validation JWT) → Controller (routing) → Service
Vue Physique	Deploiement : Navigateur Web (HTML/CSS/JS) → Serveur Spring Boot port 8081 (Docker sur Render)
Vue Scenarios	Les 15 User Stories couvrant les 3 acteurs reparties sur 3 sprints Scrum de 2 semaines chacun

6. Justification des Design Patterns

6.1 Pattern Singleton

Element	Detail
Classe	DatabaseConfig (@Configuration Spring)
Probleme	Eviter la creation de multiples instances de configuration
Solution	Spring IoC garantit une seule instance de chaque bean @Configuration
Justification	Optimise les ressources et garantit la coherence de la configuration
Principe SOLID	Single Responsibility Principle

6.2 Pattern Observer

Element	Detail
Classes	Observable (interface), Observateur (interface), OuvrageDisponibleObservable (impl)
Probleme	Notifier automatiquement les etudiants quand un ouvrage reserve devient disponible
Solution	Le retour d'un ouvrage declenche une notification a tous les observateurs inscrits
Justification	Decouple le systeme de notification du systeme d'emprunt
Principe SOLID	Open/Closed et Dependency Inversion

6.3 Pattern Factory

Element	Detail
Classe	UtilisateurFactory
Probleme	Creer le bon type d'utilisateur sans connaitre le type exact a l'avance
Solution	La factory cree Etudiant, Bibliothecaire ou Administrateur selon le role fourni
Justification	Centralise la creation d'objets et facilite l'ajout de nouveaux types
Principe SOLID	Single Responsibility et Open/Closed

6.4 Pattern Strategy

Element	Detail
Interface	CalculPenalite
Implementations	PenaliteStandard (100 FCFA/jour), PenaliteBoursier (50 FCFA/jour)
Probleme	Le calcul des penalites varie selon le type d'etudiant
Solution	Chaque strategie implemente l'interface CalculPenalite avec sa logique propre
Justification	Changement d'algorithme sans modifier EmpruntService. Respects Open/Closed
Principe SOLID	Open/Closed Principle